

14-Feb-2026

Code karna + Padhai karna

node <filename> **SHREE**

run the Command in terminal

Date: / / Page No. _____

JavaScript Basics

01-Console

```
const clue1 = "Muddy Footprint near the window";
```

- memory ke under ek space reserve ki or me us space ko me change nahi karungi.

What is the Console.log

- Ek print statement hai.

- print out the values I have in my memory space.

```
console.log("Clue found:", clue1)
```

output

clue found: Muddy footprint near the window

Note:-

CamelCase = is way of writing phrases without spaces, where the first letter of each word is capitalized, except for the first letter of the entire command word, which may be either upper or lower case.

It improves code readability.

ex,

~~Console.log~~

```
const suspectName = "Dipali";
```

```
const susAge suspectAge = 20;
```

```
console.log("Suspect: ", suspectName, " | Age: " + suspectAge)
```

output

Suspect: Dipali | Age: 20

- Console.warn()

a method in JavaScript is used to output a warning message to the web console.

ex,

```
console.warn("Warning: fingerprint evidence detected")
```

Console.warn & Console.error those are just logging, just shows the in different colors sometimes.
For Convex Iteration.

SHREE

Date: / / Page No. _____

- Console.error()

this method in JavaScript is used to output an error message to the web console, typically displayed in red text with an error icon.

ex,

```
Console.error("errorMessage: Wrong password");
```

- array of objects

it stores an a single variable name,

it stores an ordered collection of multiple objects under a single variable name, with each object accessible via a zero-based index

ex,

```
const evidencelog = [
```

objects {
0 {id: 1, item: "Muddy footprint", Location: "Window sill"},
1 {id: 2, item: "Broken glass", location: "Living room"},
2 {id: 3, item: "Red fiber stand", Location: "Door handle"},
];

- Console.table()

this method is a debugging tool that displays tabular data (arrays or objects) in an organized, readable table format within the developer console.

- Console.group()

this method in JavaScript is used to organize log messages into a collapsible, hierarchical group within the developer console.

ex,

```
Console.group("Grouped logs")
```

```
Console.log("My log 1")
```

```
Console.log("My log 2")
```

```
Console.log("My log 3")
```

```
Console.groupEnd()
```

- easily collapses and expands
- Rarely used, used more often before jQuery

note: Useful for controller and routes

output

Group Starts

My log 1

My log 2

My log 3

- Console.time()

it used to start a timer that tracks the duration of an operation. Its used to measuring code performance.

ex,

```
Console.time("Total time takes");
```

```
let dnaMatches = 0;
```

```
for (let i = 0; i < 1-00-000; i++) {
```

```
    dnaMatches++;
```

```
}
```

```
Console.timeEnd("Total time takes");
```

Output

total time takes : 0.945ms

Pointer

1-00-000 (Does this affect the performance?)

using underscores as numeric separators does not affect the performance. the 1-00-000 and 100000 both are same it introduced to improve readability. During runtime, JavaScript ignores underscores ensuring they do not affect mathematical operations.

- Console.Count()

this method is a utility for logging the number of times it has been called with specific label. it useful for tracking function executions, loop iterations, or any repeated code block during debugging.

Use
case

- This is all are useful for debugging the code.
- memory me kya hai, variable me kya declare kar
rahi hu, process time ke bad variable ke value
kya hai

SHREE

Date: / / Page No. _____

ex,

```
Console.count("chaicode")
```

```
Console.count("chaicode")
```

```
Console.count("chaicode")
```

Output

```
chaicode:1
```

```
chaicode:2
```

```
chaicode:3
```

02-Variable

~~Deprecate~~ Deprecate

any element, whether software, hardware, model or
practice, that is currently available but is no longer
recommended.

```
var shipName = "The Amber";  
Console.log("ShipName:", shipName)
```

```
let crewCount = 12;
```

```
Console.log("Crew count:", crewCount);
```

```
crewCount = 14
```

- if you want to change the value in future.

```
Const captainName = "Jack Sparrow";
```

```
Console.log("Captain Name:", captainName);
```

```
captainName = "Dipali";
```

you cannot change the
value of const variable

- Scope of const are block level

- Always use const when you need to change value
in future then use let.

- Var is not block scoped, when you declared inside loops, if
statement not completed but can be accessed outside it,
which can lead to the bug.

Scope difference

ex.

- (Khatrnak example don't use var variable)

- (Batma bhar aa jayegi)

```
if (true) {
```

```
  var leakyTreasure = 'Gold coins';  
}
```

```
for (var i = 0; i < 10; i++)
```

```
  console.log (leakyTreasure);
```

Naming Variable

Rules:-

- let Shipspeed = 22

(camelcarry more readable.)

- let _privateLog = "secret" (underscore)

- let MONGODB_URI = "mongodb://localhost:27017" (environment variable)

- let \$name = "Dipali" (valid)

Block Scope

- variables declared with let and const inside a code block are "block-scoped" meaning they are only accessible within that block.

Not Block Scope

- Variables declared with the var keyword, inside a {} block; can be accessed from outside the block.

Manipulate Data in Const

```
Const treasureChest = {
```

```
  gold: 100,
```

```
  rubies: 50,
```

```
  maps: 2
```

```
}
```

mutating content is allowed.

```
treasureChest.gold = 150; // Content manipulate, property change
```

```
treasureChest = { gold: 50 }; // new reference not allowed
```

Complete new reference change

Similar behavior in JavaScript

```
Const crewRoster = ["Alok", "Abhinav", "Tanish"];
```

```
crewRoster.push("Viraj"); ← add the name in the last
```

```
crewRoster[0] = "Shubham"; ← remove the value in 0 index and add shubham.
```

```
crewRoster = ["Someone"];
```

error, you can't change the reference

0-3-Data Types

There are two types of data type 1) Primitive and 2) Non-primitive data type

```
Const weaponName = "Flame Sword";
```

```
Consol.log("Weapon:" weaponName, "Type:");
```

used to find out which data type → typeof weaponName
The datatype is string

```
Const attackPower = 75; → Number
```

```
Const attackUpgrade = 1.5
```

```
Const attackNumber = 75n → BigInt
```

arbitrary large Integer

`Const isLoggedIn = true;` → Boolean ^{2 values}
true / false

`let bonusEffect;` → undefined

`let CourseStatus = null;` → null (value is intentionally absent)
ex,

`let weatherApiResponse = null;` → `typeof` is gonna print
`Console.log(weatherApiResponse);` Object (Interview question)
~~legacy~~ → (Data `typeof null` is null) it's backward compatibility
bug, it is quirk of JavaScript

`Const uniqueRuneId = symbol("rune-of-fire")`
`console.log("Rune:", uniqueRuneId.toString());`

Symbol unique and immutable.

`toString` is a built in function that converts a value or
an object to its string representation
`typeof` is symbol is symbol

What is the meaning of Rune

- Rune is a character of an ancient Germanic alphabet, related to the Roman alphabet.
- and in this code rune is met-metaphorical name of a symbol

ex,

✖, ✚, ✕ — Rune Symbols
Carriage energy Good Luck

There are 7 primitive Data-types in JavaScript

- String
 - Number
 - Boolean
 - BigInt
 - Symbol
 - Undefined
 - null
- Null! - Intentional absence of a value
or an empty state.
- undefined! - It's mean variable
has been declared but not
been assigned a value.

Non Primitive Data typeObject

```

Const heroStatus = {
    name: "Deepak";
    level: 12;
    Class: "Ranger";
}

```

```

Console.log("Hero: " + heroStatus, " | type: ",
    typeof heroStatus);

```

Output

```

Hero: { name: "Deepak", level: 12, class: 'Ranger' }
 | type: Object

```

typeof object is object

Array

```

Const inventory = ["Flame sword", "Health Potion",
    "shield"];

```

```

Console.log("Inventory: " + inventory, " | type: ",
    type inventory);

```

Output

```

Inventory: ["Flame sword", "Health Potion", "Shield"]

```

 | type object

typeof array is object

Function

```

Function castSpell() {
    return "Fireball";
}

```

```

Console.log("Spell Type: " + typeof castSpell);

```

typeof castSpell is
function

Primitive = Copy by value
non-Primitive = Copy by reference

SHREE

note: ↑

Date: / / Page No. _____

Console.log (type of "charCode"); ⇒ String
Console.log (typeof 42); ⇒ number
Console.log (typeof 42n); ⇒ BigInt
Console.log (type of true); ⇒ Boolean
Console.log (type of undefined); ⇒ undefined
Console.log (type of null); ⇒ Object
Console.log (type of Symbol()); ⇒ Symbol
Console.log (type of {}); ⇒ Object
Console.log (typeof []); ⇒ Object
Console.log (typeof function () {}); ⇒ Function

How to copy the value

```
const originalSword = {  
  name: "flame sword",  
  damage: 75,  
  typeOfw: "fire",  
};
```

} value of damage
also change

Reference ka issue hota hai

```
const clonesword = originalSword;
```

```
clonesword.damage = 100;
```

(copy me changes kien
to original may be changes
change)

```
console.log ("Original sword: ", originalSword.damage);
```

Don't use this way to copy data values

```
const armorOriginal = {
```

```
  name: "Iron Plate",
```

```
  defence: 80,
```

```
  buff: {
```

```
    fire: 10,
```

```
  },
```

```
};
```

this defend

as nested Object

→ This means changes to the nested structures in the copy will also affect the original.

shallow clone (fancy word)

```
const armorCopy = { ...armorOriginal }
```

└ spread operator

(It's spread the all properties then combine it)

→ you get new copy.

```
armorCopy.buff.fire = 90;
```

It's only allows you to change in copy not in main.

use when you don't have nested structure.

another example using structuredClone

```
const potionOriginal = {
```

```
  name: "Health",
```

```
  effects: {
```

```
    heal: 40,
```

```
    mana: 30
```

```
  }
```

```
}
```

(fancy word)

└ deep clone

```
const potionCopy = structuredClone(potionOriginal);
```

↓
It's allows you to copy nested properties

StructuredClone()

It is a function in JavaScript and it is way to create a deep copy of an object or value

→ It creates a new object and recursively copies all nested objects and arrays, ensuring the clone is completely independent of the original

Note:- `typeof null === "object";` it's legacy bug

check way value

```
let value = null;
```

```
console.log(value); Output null
```

Note:- check array is array or not (methods)

Array.isArray() $\Rightarrow$ It returns true/false **SHREE**

because type of array is object

Date: / / Page No. _____

04 - numbers

Const crewMembers = 40

Const fuelTons = 142.42

Const light-speed = 299888999

In JavaScript integers and float both are numbers

It's for readability

special numbers in javascript for DSA / computational programming

Const infiniteRange = Infinity

Const negativeInfiniteRange = -Infinity

NAN (Not A Number)

Const notANumber = NAN

- It's a invalid mathematical operation or a failed attempt to convert a non-numeric value into a number

- using (typeof NAN) will return "number"

- NAN is unique, it's not equal to any other value, ex. $NAN === NAN$ is false

"5"

$\Rightarrow$ Can convert into an number

"Deepak" $\Rightarrow$ Doesn't convert into an number shows NAN

- NAN is just a output

Const log(1/0);

Const +

Console.log(1/0);

$\Rightarrow$ Infinity

Console.log(-1/0);

$\Rightarrow$ -Infinity

DSA Purpose:- math operation

SHREE

Date: / / Page No. _____

Console.log (Number.Max_SAFE_INTEGER);

- 9007199254740991

Console.log (Number.Min_SAFE_INTEGER);

- 9007199254740991

- Console.log (Number.EPSILON);

- It is a static property that represents the difference between 1 and the smallest floating-point number greater than 1 that can be represented.

- The value is 2^{-52} (2.220446049.....)

- Console.log (Number.isInteger());

- Using this method you can check if a number is an integer or not. It returns true & false value.

- Console.log (Number.isFinite());

- In this method you can check if a value is finite number or not. It returns true only if the value is of the type number and is a finite number.

- Console.log (Number.isNaN());

- In this method you can check if a value is not a number or not. It returns true only if the value is of the type number and the value NaN.

- Console.log (Number.

Parsing

or

Const fuelReading = "142.75 tons";

Const sectorCode = "0XA3"; ← Hex code (also

Const countDown = "007";

represented as
pound sign £)

Console.log (parseInt (countDown));

value is 163.

Console.log (parseInt ("111", 2));

- Console.log(parseInt())

it parses a string argument and returns an integer (whole number) of a specified radix (base).

- Console.log(parseFloat())

it parses a string argument and returns a floating point number.

Math methods

Const thrustforce = 4.567;

Console.log(Math.round(thrustforce)); $\Rightarrow 5$

Console.log(Math.floor(thrustforce)); $\Rightarrow 4$

Console.log(Math.ceil(thrustforce)); $\Rightarrow 5$

Console.log(Math.trunc(thrustforce)); $\Rightarrow 4$

- Math.round()

In this method it returns the value of a number rounded to the nearest integer.

ex:

Math.round(2.49) $\Rightarrow 2$

Math.round(2.87) $\Rightarrow 3$

- Math.floor()

this method is used to round a number down to the nearest integer that is less than or equal to original number.

ex

Math.floor(5.9) $\Rightarrow 5$

Math.floor(-3.7) $\Rightarrow -4$

Math.floor(10) $\Rightarrow 10$

- Math.ceil()

It's rounds a number up to the nearest integer greater than or equal to the given number

ex,

`Math.ceil(4.2)` $\Rightarrow$ 5

`Math.ceil(4.9)` $\Rightarrow$ 5

`Math.ceil(-2.7)` $\Rightarrow$ -2 (-2 is greater than -2.7)

`Math.ceil(2+3.8)` $\Rightarrow$ 6 (round up the result of 5.8)

- Math.trunc()

It's returns the integer part of a number by removing its fractional digits.

ex,

`Math.trunc(13.37)` $\Rightarrow$ 13

`Math.trunc(42.84)` $\Rightarrow$ 42

`Math.trunc("420.56")` $\Rightarrow$ 420 (string is implicitly converted into a number)

`Math.trunc(-1.123)` $\Rightarrow$ -1

- Math.min()

It's returns the smallest of the numbers given as input parameters.

ex,

`const temps = [-120, 43, 56, -23];`

~~`console.log(Math.min(temps));`~~

`console.log(Math.min(...temps));` $\Rightarrow$ -120

↑ Spread operator for accessing the data in arrays

- `Math.min()` method does not accept an array directly. you must use the spread operator to expand the array elements as individual arguments.

- Math.max()

It returns the largest of zero or more numbers provided as input parameters.

ex.

```
const temps = [-120, 43, 56, -23]
console.log(Math.max(...temps)) // => 56
```

Similar to the `math.min` you also have to use spread operator in the `math.max()` to.

Interview question

```
console.log(0.1 + 0.2); // => 0.300000004
```

```
console.log(0.1 + 0.2 === 0.3); // => false
```

```
function almostEqual(a, b) {
  return Math.abs(a - b) < Number.EPSILON;
}
```

If number is negative, positive or zero it converts them into positive.

```
console.log(almostEqual(0.1 + 0.2, 0.3)); // => true
```

`===` is a strict equality operator and it checks the extreme mathematical condition. You think `0.1 + 0.2` is `0.3` but you are wrong. We humans think it is `0.3` but computer sees it `0.3000000004`. So that's why they are not equal.

And the 3rd `(0.1 + 0.2, 0.3)` is true because `0.3000000004` is less than `Number.EPSILON`. The value of `EPSILON` is 2^{-52} and using `Math.abs` it converts all the values in positive. That's why it's true.

(Maximum values are store in string format)

0.5-String

Three ways to define strings

- `const codeName = "Shadow fox";`

- `const backupName = String ("Night Owl");`

↳ `String()` constructor

- `const templateName = 'Agent ${codeName}';`

↳ String interpolation

- `${}` placeholders

- String are immutable

ex

`let intercepted = "HELLO";`

`replace 'H' with intercepted = 'J';` // silent fail

`'J'`

`console.log (intercepted);` ⇒ print HELLO

String properties and methods

1) `.length`

It used to determine the number of items or characters in various data structure.

ex,

`const secretCode = "OMEGA-7";`

`console.log (secretCode.length);` ⇒ 7

2) `.charAt()`

It's returns a new string consisting of the single character at the specified index within a string the index starts with 0 and the second is 1

ex.

```
const secretCode = "OMEGA-7";
```

```
console.log(secretCode.charAt(1));  $\Rightarrow$  M
```

```
console.log(secretCode.charAt(99));  $\Rightarrow$  empty string
```

```
console.log(secretCode[99]);  $\Rightarrow$  undefined
```

3) `.at()`

it works same as `.charAt()` but it also supports the negative indexing.

ex.

```
const secretCode = "OMEGA-7";
```

```
console.log(secretCode.at(-1));  $=$  7
```

$\Rightarrow$ because -1 returns

the last element.

```
console.log(secretCode.at(-2));  $\Rightarrow$  -
```

second last element.

4) `toUpperCase()`

Converts the entire string to uppercase letters.

ex.

```
const rawTransmission = "The Eagle has LANDED";
```

```
console.log(rawTransmission.toUpperCase());
```

$\Rightarrow$ THE EAGLE HAS LANDED

5) `toLowerCase()`

Converts the entire string to lowercase letters.

ex.

```
const rawTransmission = "The Eagle has LANDED";
```

```
console.log(rawTransmission.toLowerCase());
```

$\Rightarrow$ the eagle has landed

3) slice (start, end)

It extracts a section of a string and returns it as a new string, without modifying the original.

ex,

```
const myName = "My name is muku";  
console.log(myName.slice(10, 15));  $\Rightarrow$  muku
```

4) trim()

It removes whitespace from both the beginning and the end of a string.

ex

```
const myNameIs = " My name is Kuku "  
console.log(myNameIs.trim());  $\Rightarrow$  My name is Kuku
```

5) includes()

This method performs a case-sensitive search and returns a boolean value true or false.

ex,

```
const pet = "my pet name is muku";  
console.log(pet.includes("muku"));  $\Rightarrow$  true
```

6) split()

This method is used to divide a string into an ordered list of substrings and return them as an array.

The original string is not modified.

ex,

```
const myPet = "my pet is rabbit";  
console.log(myPet.split());  $\Rightarrow$  ['my', 'pet', 'is', 'rabbit']
```

Split example

```
const Orders = "move-north|hold-position|extract-vip"
```

```
OrderList = Orders.split("|");
```

```
console.log("Split", OrderList);
```

⇒ ["move-north", "hold-position", "extract-vip"]

Date
Page

7) Join()

It combines all elements of an array into a single string

ex

```
const Fruits = ['Apple', 'Banana', 'Orange'];
```

```
console.log(Fruits.join());
```

⇒ Apple, Banana, Orange.

8) replace()

This method returns a new string with value(s) replaced.

ex.

```
const good = "Muku is a rabbit and he is good pet";
```

```
console.log(good.replace("muku", "Kuku"));
```

⇒ Kuku is a rabbit and he is good pet.

9) repeat()

It returns a new string with a specified number of the original string.

ex.

```
const repeatName = "Kuku";
```

```
console.log(repeatName.repeat(3));
```

⇒ Kuku Kuku Kuku

10) Indexof()

It returns the zero-based index of the first occurrence or -1 if the value is not found.

ex.

```
const home = "Welcome to home";
```

```
console.log(home.indexOf("home"));
```

⇒ 11

// tagged template literal may not result in string; it can be used with a custom tag function to perform whatever operations you want on the different parts of the template literal.

Date _____
Page _____

11) `padStart (Length, string)`
`padEnd (Length, string)`
pads the current string from the start or end with another string until it reaches a given length.
ex;

```
const missionNumber = '42'  
console.log (missionNumber.padStart (6, "0"));  
console.log (missionNumber.padEnd (6, "0"));  
⇒ 000042
```

420000

Conditional block in template literal

```
const profile = '  
$ { checker ? "true-value" : "false-value" }'
```

Ternary operator

Interview question

```
console.log (void "Me");
```

output ⇒ undefined

- void means nothing
- void always evaluate expression and return undefined.
- It use for type safety when function returns nothing then use (void)

Note:- What will be the changes in memory when it is undefined and null?

Undefined is the default value of variable that has been declared but not yet initialized. i.e. value not yet assigned.
null it is empty or has no value.

memory clearing = In given example we use ~~How to demolish value completely~~ `generalStore = null` to break to an object. This allows the garbage collector to reclaim the memory used by that object. Setting it to undefined achieves the same result but `null` is for ~~this~~ reference is now intentionally empty.

Hidden classes: Sometime we switch a property between number and null, engine might be struggle to understand hidden classes. Consistency in which empty value you use can actually improve memory performance over time.

How to demolish value completely.

```
const let generalStore = { name: "Kiran", goods: 2 }  
Console.log (generalStore);  
generalStore = null;  
Console.log (generalStore);
```

```
switch (chosenPath) {
```

```
    case "right":
```

```
        console.log("User selected right");
```

```
        break;
```

```
    case "top":
```

```
        console.log("User selected top");
```

```
        break;
```

```
    case "left":
```

```
        console.log("User selected - left");
```

```
        break;
```

```
    default: "this default value";
```

```
}
```